

DIGITALITZACIÓ D'UN TALLER MECÀNIC CONNECTAT

Memòria del projecte

Mòdul: 1665 Digitalització Aplicada als Sectors Productius

Alumnes: Víctor Díaz i Marc Tiscar

Professor: David González

Data de lliurament: 26/05/2026

Web + API Flask + MariaDB + Docker + scripts de comprovació

Projecte provat amb captures reals de funcionament

Resum del projecte

En aquesta memòria s'explica el desplegament d'una aplicació senzilla per a un taller mecànic. La solució permet consultar vehicles i registrar cites des d'una pàgina web. Les dades es guarden en una base de dades MariaDB i la comunicació es fa a través d'una API en Flask. Tot s'ha executat amb Docker Desktop.

Índex

1. Introducció	3
2. Anàlisi inicial	4
3. Estructura del projecte	5
4. Model de dades	6
5. Desenvolupament de la solució	8
6. Infraestructura Docker	12
7. Proves de funcionament	14
8. Scripts i còpies de seguretat	17
9. Problemes trobats i solucions	19
10. Conclusions i annexos	21

1. Introducció

Aquest projecte consisteix a digitalitzar una part bàsica de la gestió d'un taller mecànic. La idea és tenir una aplicació web senzilla on es puguin consultar vehicles i crear cites de taller.

La solució està formada per tres parts principals: una base de dades MariaDB, una API feta amb Python i Flask, i una pàgina web amb HTML, CSS i JavaScript. La web és la part visible, l'API fa de pont, i la base de dades guarda la informació.

Per desplegar-ho s'ha utilitzat Docker Desktop. Això permet aixecar els serveis del projecte sense haver d'instal·lar MariaDB ni Flask directament a Windows. Cada part funciona dins del seu contenidor.

Objectiu principal

Aconseguir que una cita creada des de la web quedi registrada a la base de dades i es pugui tornar a consultar des de la mateixa web o des de l'API.

Objectius concrets

- Crear una base de dades amb clients, vehicles i cites.
- Fer una API bàsica amb Flask que treballi amb JSON.
- Preparar una web amb formulari de cita.
- Executar el projecte amb Docker Desktop.
- Fer proves reals del funcionament.
- Generar una còpia de seguretat de la base de dades.

2. Anàlisi inicial

El taller necessita una manera més ordenada de guardar clients, vehicles i cites. Amb aquesta solució, les dades no queden en paper o separades en diferents llocs, sinó centralitzades en una base de dades.

Necessitats detectades

Apartat	Necessitat	Solució aplicada
Clients	Guardar les dades bàsiques del client.	Taula clients.
Vehicles	Relacionar cada cotxe amb el seu propietari.	Taula vehicles amb idClient.
Cites	Registrar dia, hora i servei demanat.	Taula cites.
Web	Permetre demanar una cita de forma senzilla.	Formulari HTML amb JavaScript.
API	Connectar la web amb la base de dades.	Flask retornant JSON.
Docker	Executar-ho tot de forma ordenada.	docker-compose.yml amb tres serveis.

Planificació seguida

El treball s'ha fet revisant primer l'estructura del projecte. Després s'ha aixecat amb Docker Desktop i s'han comprovat les parts una a una: contenidors, web, API, creació de cites, backup i comprovació de serveis.

Fase	Treball fet
1	Revisió de carpetes i fitxers del projecte.
2	Comprovació del model de dades i del script SQL.
3	Arrencada dels contenidors amb Docker.
4	Proves de la web i de les rutes de l'API.
5	Còpia de seguretat i revisió final de captures.

3. Estructura del projecte

El projecte està separat per carpetes perquè sigui més fàcil trobar cada part. La carpeta api conté el backend, la carpeta web conté la pàgina, scripts guarda les automatitzacions i backups les còpies de seguretat.

Nombre	Estado	Fecha de modificación	Tipo	Tamaño
api	✓	26/05/2026 13:13	Carpeta de archivos	
backups	✓	26/05/2026 13:13	Carpeta de archivos	
docs	✓	26/05/2026 13:13	Carpeta de archivos	
scripts	✓	26/05/2026 13:13	Carpeta de archivos	
web	✓	26/05/2026 13:13	Carpeta de archivos	
.gitignore	✓	26/05/2026 13:13	Archivo de origen Git...	1 KB
db_schema.sql	✓	26/05/2026 13:13	SQLFile	2 KB
docker-compose.yml	✓	26/05/2026 13:13	Archivo de origen Ya...	1 KB
README.md	✓	26/05/2026 13:13	Archivo de origen M...	1 KB

Figura 1. Estructura principal del projecte vista des de l'Explorador de Windows.

Fitxers principals

Fitxer o carpeta	Funció
api/	Codi de l'API Flask.
web/	Pàgina web del taller.
scripts/	Scripts de backup i comprovació.
backups/	Còpies de seguretat generades.
docs/	Documentació i diagrama E/R.
db_schema.sql	Script de creació de la base de dades.
docker-compose.yml	Fitxer per aixecar els contenidors.

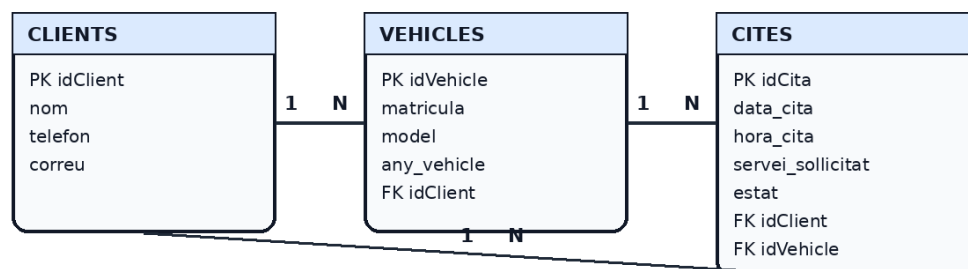
Estructura resumida

```
projecte_taller_ASIX1/
├── api/
├── web/
├── scripts/
├── backups/
├── docs/
├── db_schema.sql
├── docker-compose.yml
└── README.md
```

4. Model de dades

La base de dades s'anomena taller. Té tres taules principals: clients, vehicles i cites. La relació és bastant directa: un client pot tenir vehicles, i una cita queda associada a un client i a un vehicle.

Diagrama E/R - Taller Mecànic Connectat



Relacions:

- Un client pot tenir diversos vehicles.
- Un vehicle pot tenir diverses cites.
- Una cita queda associada a un client i a un vehicle.

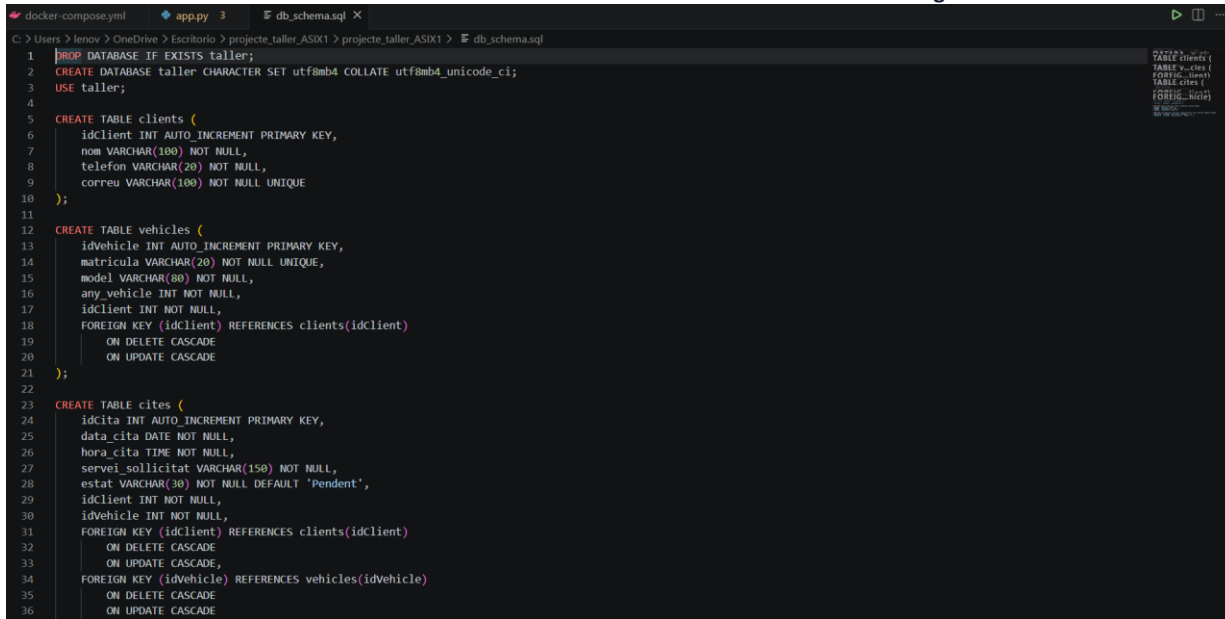
Figura 2. Diagrama E/R de la base de dades del taller.

Taules de la base de dades

Taula	Camps principals	Explicació
clients	idClient, nom, telefon, correu	Guarda les dades dels clients.
vehicles	idVehicle, matricula, model, any_vehicle, idClient	Guarda els vehicles i els relaciona amb el client.
cites	idCita, data_cita, hora_cita, servei_sollicitat, estat, idClient, idVehicle	Guarda les cites demanades al taller.

Script SQL

El fitxer db_schema.sql crea la base de dades i defineix les taules. També inclou les claus foranes per relacionar clients, vehicles i cites.



```

1 DROP DATABASE IF EXISTS taller;
2 CREATE DATABASE taller CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
3 USE taller;
4
5 CREATE TABLE clients (
6     idClient INT AUTO_INCREMENT PRIMARY KEY,
7     nom VARCHAR(100) NOT NULL,
8     telefon VARCHAR(20) NOT NULL,
9     correu VARCHAR(100) NOT NULL UNIQUE
10 );
11
12 CREATE TABLE vehicles (
13     idVehicle INT AUTO_INCREMENT PRIMARY KEY,
14     matricula VARCHAR(20) NOT NULL UNIQUE,
15     model VARCHAR(80) NOT NULL,
16     any_vehicle INT NOT NULL,
17     idClient INT NOT NULL,
18     FOREIGN KEY (idClient) REFERENCES clients(idClient)
19     ON DELETE CASCADE
20     ON UPDATE CASCADE
21 );
22
23 CREATE TABLE cites (
24     idcita INT AUTO_INCREMENT PRIMARY KEY,
25     data_cita DATE NOT NULL,
26     hora_cita TIME NOT NULL,
27     servei_sollicitat VARCHAR(150) NOT NULL,
28     estat VARCHAR(30) NOT NULL DEFAULT 'Pendent',
29     idClient INT NOT NULL,
30     idVehicle INT NOT NULL,
31     FOREIGN KEY (idClient) REFERENCES clients(idClient)
32     ON DELETE CASCADE
33     ON UPDATE CASCADE,
34     FOREIGN KEY (idVehicle) REFERENCES vehicles(idVehicle)
35     ON DELETE CASCADE
36     ON UPDATE CASCADE

```

Figura 3. Fitxer db_schema.sql amb la creació de les taules principals.

5. Desenvolupament de la solució

5.1 API amb Python i Flask

L'API és la part que rep les peticions de la web. Quan la web necessita clients, vehicles o cites, fa una petició a l'API. L'API consulta MariaDB i retorna les dades en format JSON.

Ruta	Mètode	Funció
/	GET	Comprovar que l'API respon.
/health	GET	Comprovar API i base de dades.
/clients	GET	Llistar clients.
/vehicles	GET	Llistar vehicles amb el client.
/appointments	GET	Llistar les cites registrades.
/appointments	POST	Crear una cita nova.

```

45
46 @app.route("/", methods=["GET"])
47 def home():
48     return jsonify({"message": "API del taller funcionant"})
49
50
51 @app.route("/health", methods=["GET"])
52 def health():
53     try:
54         db = connect_db()
55         db.close()
56         return jsonify({"api": "ok", "database": "ok"})
57     except Exception as error:
58         return jsonify({"api": "ok", "database": "error", "detail": str(error)}), 500
59
60
61 @app.route("/clients", methods=["GET"])
62 def get_clients():
63     clients = query_db("SELECT * FROM clients ORDER BY idClient")
64     return jsonify(clients)
65
66
67 @app.route("/vehicles", methods=["GET"])
68 def get_vehicles():
69     sql = """
70     SELECT v.idVehicle, v.matricula, v.model, v.any_vehicle,
71           c.idClient, c.nom AS client
72     FROM vehicles v
73     INNER JOIN clients c ON v.idClient = c.idClient
74     ORDER BY v.idVehicle
75     """
76     vehicles = query_db(sql)
77     return jsonify(vehicles)
78
79
80 @app.route("/appointments", methods=["GET"])

```

Figura 4. Codi de l'API Flask amb les rutes /health, /clients, /vehicles i /appointments.

5.2 Web amb HTML/CSS/JS

La web permet seleccionar un client i un vehicle, indicar una data, una hora i escriure el servei sol·licitat. Quan s'envia el formulari, JavaScript fa una petició fetch a l'API.

```

1  const API_URL = "http://localhost:5000";
2
3  async function carregarClients() {
4      const resposta = await fetch(`${API_URL}/clients`);
5      const clients = await resposta.json();
6      const select = document.getElementById("idClient");
7      select.innerHTML = "";
8
9      clients.forEach(client => {
10         const opcio = document.createElement("option");
11         opcio.value = client.idClient;
12         opcio.textContent = `${client.idClient} - ${client.nom}`;
13         select.appendChild(opcio);
14     });
15 }
16
17 async function carregarVehicles() {
18     const resposta = await fetch(`${API_URL}/vehicles`);
19     const vehicles = await resposta.json();
20
21     const select = document.getElementById("idVehicle");
22     const div = document.getElementById("llistaVehicles");
23     select.innerHTML = "";
24     div.innerHTML = "";
25
26     vehicles.forEach(vehicle => {
27         const opcio = document.createElement("option");
28         opcio.value = vehicle.idVehicle;
29         opcio.textContent = `${vehicle.idVehicle} - ${vehicle.matricula} (${vehicle.model})`;
30         select.appendChild(opcio);
31     });

```

Figura 5. Codi JavaScript on es veu l'ús de fetch per carregar clients i vehicles des de l'API.

6. Infraestructura Docker

El fitxer docker-compose.yml defineix els contenidors del projecte. S'han utilitzat tres serveis: db per a MariaDB, api per a Flask i web per a la pàgina del taller.

```

1 services:
2   db:
3     image: mariadb:11
4     container_name: taller_db
5     restart: unless-stopped
6     environment:
7       MYSQL_ROOT_PASSWORD: example
8       MYSQL_DATABASE: taller
9     volumes:
10      - db_data:/var/lib/mysql
11      - ./db_schema.sql:/docker-entrypoint-initdb.d/db_schema.sql:ro
12     networks:
13       taller_net:
14         aliases:
15           - db.taller.local
16
17   api:
18     build: ./api
19     container_name: taller_api
20     restart: unless-stopped
21     ports:
22       - "5000:5000"
23     depends_on:
24       - db
25     environment:
26       DB_HOST: db
27       DB_USER: root
28       DB_PASSWORD: example
29       DB_NAME: taller
30     networks:
31       taller_net:
32         aliases:
33           - api.taller.local
34
35   web:
36     build: ./web
37     container_name: taller_web
38     restart: unless-stopped
39     ports:
40       - "8080:80"
41     depends_on:
42       - api
43     networks:
44       taller_net:

```

Figura 6. Fitxer docker-compose.yml amb els serveis db, api i web.

Contenidors i ports

Servei	Contenidor	Port / ús
Base de dades	taller_db	MariaDB, port intern 3306.
API	taller_api	Port 5000 per consultar les rutes Flask.
Web	taller_web	Port 8080 per obrir la pàgina.

Docker Desktop

Un cop executat docker compose up --build, els contenidors apareixen a Docker Desktop. Això permet veure ràpidament si els serveis estan en marxa.

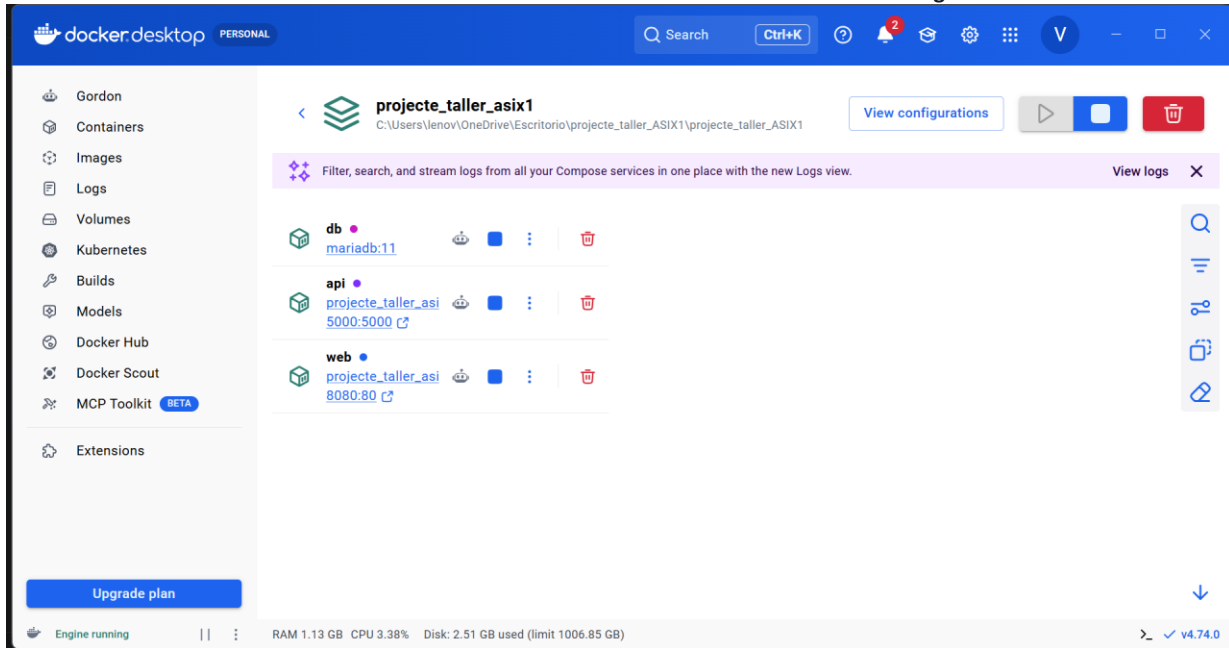


Figura 7. Docker Desktop amb els tres contenidors del projecte funcionant.

7. Proves de funcionament

7.1 Comprovació per terminal

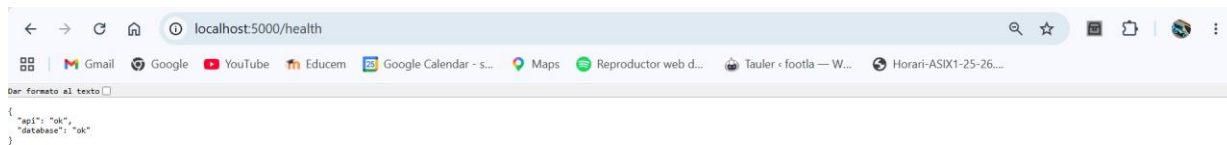
La primera comprovació important és docker ps. Aquesta ordre mostra que els tres contenidors estan actius i quins ports tenen publicats.

```
PS C:\Users\lenov\OneDrive\Escritorio\projecte_taller_ASIX1\projecte_taller_ASIX1> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
5b346fbfa5b9   projecte_taller_asix1-web   "/docker-entrypoint..." 2 minutes ago  Up 2 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
df64b1782a80   projecte_taller_asix1-api    "python app.py"           2 minutes ago  Up 2 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
d4d2e8a954b1   mariadb:11      "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes  3306/tcp
taller_db
```

Figura 8. Comprovació amb docker ps dels contenidors taller_web, taller_api i taller_db.

7.2 Comprovació de salut de l'API

La ruta /health serveix per revisar ràpidament si l'API respon i si la connexió amb la base de dades funciona.

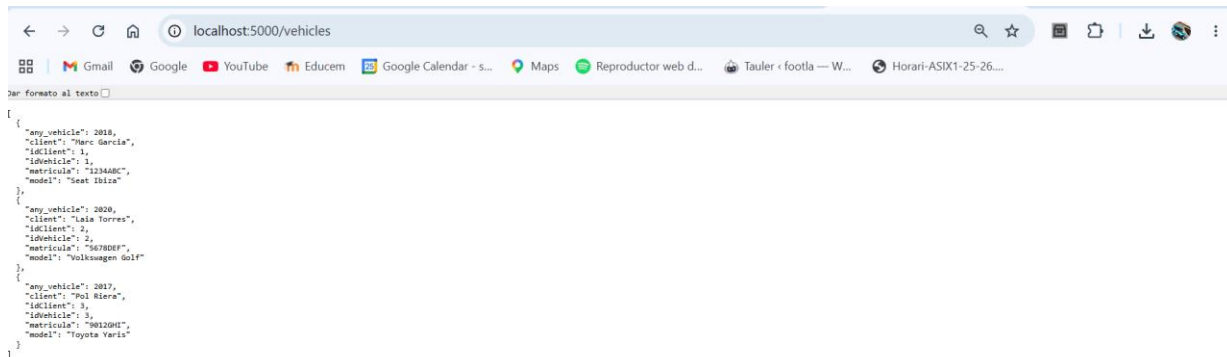


```
{
  "api": "ok",
  "database": "ok"
}
```

Figura 9. Ruta /health retornant api ok i database ok.

7.3 Consulta de vehicles

La ruta /vehicles retorna els vehicles guardats a la base de dades. Es veu la matrícula, el model, l'any i el client associat.



```
[
  {
    "any_vehicle": 2018,
    "client": "Marc Garcia",
    "id_client": 1,
    "id_vehicle": 1,
    "matricula": "1234ABC",
    "model": "Seat Ibiza"
  },
  {
    "any_vehicle": 2020,
    "client": "Laia Torres",
    "id_client": 2,
    "id_vehicle": 2,
    "matricula": "5678DEF",
    "model": "Volkswagen Golf"
  },
  {
    "any_vehicle": 2017,
    "client": "Poi Riera",
    "id_client": 3,
    "id_vehicle": 3,
    "matricula": "9012GHI",
    "model": "Toyota Yaris"
  }
]
```

Figura 10. Ruta /vehicles retornant dades en format JSON.

7.4 Consulta de cites

També s'ha comprovat /appointments. Aquesta ruta retorna totes les cites registrades, incloent client, vehicle, data, hora, estat i servei sol·licitat.

```
{
  {
    "client": "Marc Garcia",
    "data_cita": "2026-05-20",
    "estat": "Pendent",
    "hora_cita": "09:30",
    "id_cita": 1,
    "matricula": "1234ABC",
    "model": "Seat Ibiza",
    "servei_sol·licitat": "Canvi d'oli"
  },
  {
    "client": "Lola Torres",
    "data_cita": "2026-05-21",
    "estat": "Pendent",
    "hora_cita": "11:00",
    "id_cita": 2,
    "matricula": "5678DEF",
    "model": "Volkswagen Golf",
    "servei_sol·licitat": "Revisió general"
  },
  {
    "client": "Marc Garcia",
    "data_cita": "2026-05-22",
    "estat": "Pendent",
    "hora_cita": "16:37",
    "id_cita": 3,
    "matricula": "1234ABC",
    "model": "Seat Ibiza",
    "servei_sol·licitat": "Canvi d'oli"
  }
}
```

Figura 11. Ruta /appointments retornant les cites registrades.

7.5 Web del taller

La web funciona a http://localhost:8080. En la captura es veu el formulari per demanar cita i el llistat de cites registrades.

Taller Mecànic Connectat
Gestió digital de clients, vehicles i cites

Demanar cita

Client
1 - Marc Garcia

Vehicle
1 - 1234ABC (Seat Ibiza)

Data
dd/mm/aaaa

Hora
--:--

Servei sol·licitat
Ex: canvi d'oli

Enviar cita

Cites registrades

Actualitzar cites

2026-05-13 - 18:44
Client: Pol Riera
Vehicle: 1234ABC (Seat Ibiza)
Servei: Canvi de rodes
Estat: Pendent

2026-05-20 - 09:30
Client: Marc Garcia
Vehicle: 1234ABC (Seat Ibiza)
Servei: Canvi d'oli
Estat: Pendent

2026-05-21 - 11:00
Client: Lola Torres
Vehicle: 5678DEF (Volkswagen Golf)
Servei: Revisió general
Estat: Pendent

2026-05-22 - 16:37
Client: Marc Garcia
Vehicle: 1234ABC (Seat Ibiza)
Servei: Canvi d'oli
Estat: Pendent

2026-05-26 - 12:42
Client: Lola Torres
Vehicle: 9876GHI (Toyota Yaris)
Servei: Fallo motor

Figura 12. Web del taller funcionant a localhost:8080 amb les cites carregades.

7.6 Creació d'una cita des de la web

La prova més important ha estat crear una cita des del formulari. Després d'enviar-la, apareix al llistat de cites. Això confirma que la comunicació web, API i base de dades funciona.

Figura 13. Cita creada correctament des de la web i mostrada en el llistat

8. Scripts i còpies de seguretat

A part de la web i l'API, també s'han fet proves relacionades amb l'automatització. La més important és la còpia de seguretat de la base de dades.

Backup de MariaDB

La còpia s'ha generat amb mysqldump dins del contenidor taller_db. Després s'ha comprovat la carpeta backups per veure que s'ha creat el fitxer .sql.

```
docker exec taller_db mysqldump -uroot -pexample taller > backups/backup_taller.sql
dir backups
```

PS C:\Users\lenov\OneDrive\Escritorio\projecte_taller_ASIX1\projecte_taller_ASIX1> dir backups

Directorio: C:\Users\lenov\OneDrive\Escritorio\projecte_taller_ASIX1\projecte_taller_ASIX1\backups

Mode	LastWriteTime	Length	Name
-a---l	26/05/2026 13:23	230	backup_taller_20260526_132358.sql

Figura 14. Comprovació de la carpeta backups amb el fitxer .sql generat.

Comprovació de serveis

També es va provar el script check_services.sh. L'API i la web responen correctament. En la part de MariaDB va aparèixer un avís de WSL, però la base de dades estava activa i funcionava, com es demostra amb docker ps i amb la creació de cites.

```
PS C:\Users\lenov\OneDrive\Escritorio\projecte_taller_ASIX1_complet_ARREGLAT\projecte_taller_ASIX1> docker exec taller_d
b mysqldump -uroot -pexample taller > backups/backup_taller.sql
PS C:\Users\lenov\OneDrive\Escritorio\projecte_taller_ASIX1_complet_ARREGLAT\projecte_taller_ASIX1> dir backups
```

Directorio: C:\Users\lenov\OneDrive\Escritorio\projecte_taller_ASIX1_complet_ARREGLAT\projecte_taller_ASIX1\backups

Mode	LastWriteTime	Length	Name
-a---l	26/05/2026 12:45	258	backup_taller.sql
-a---l	26/05/2026 12:43	230	backup_taller_20260526_124342.sql
-a---l	26/05/2026 12:43	230	backup_taller_20260526_124351.sql

```
PS C:\Users\lenov\OneDrive\Escritorio\projecte_taller_ASIX1_complet_ARREGLAT\projecte_taller_ASIX1> bash scripts/check_s
ervices.sh
Comprovant contenidors del projecte...
-----
Comprovant API...
API OK: http://localhost:5000/health respon correctament

Comprovant base de dades...

The command 'docker' could not be found in this WSL 2 distro.
We recommend to activate the WSL integration in Docker Desktop settings.
For details about using Docker Desktop with WSL 2, visit:
```

Figura 15. Execució de backup i comprovació de serveis des de PowerShell/Bash.

9. Problemes trobats i solucions

Durant el projecte han sortit alguns problemes normals. S'han anotat perquè ajuden a entendre millor què s'ha comprovat i com s'ha solucionat.

Problema	Què passava	Solució
No estava clar on posar el ZIP	Docker Desktop no importa un ZIP directament.	Es va descomprimir el projecte i es va executar docker compose des de PowerShell.
/appointments donava error	Flask no podia convertir un camp d'hora de MariaDB a JSON.	Es van convertir data i hora a text abans de retornar jsonify.
Backup.sh inicial amb error	El script depenia del nom del contenidor i de l'entorn.	Es va fer el backup amb docker exec taller_db.
MariaDB KO al check_services	El script s'executava des de Bash/WSL i no detectava docker igual que PowerShell.	Es va validar MariaDB amb docker ps i amb la prova real de crear cites.

Error corregit a /appointments

Aquest va ser el problema més important. Al principi la ruta /appointments mostrava un TypeError. Després de corregir el format de les dades, la ruta ja va retornar les cites en JSON correctament.

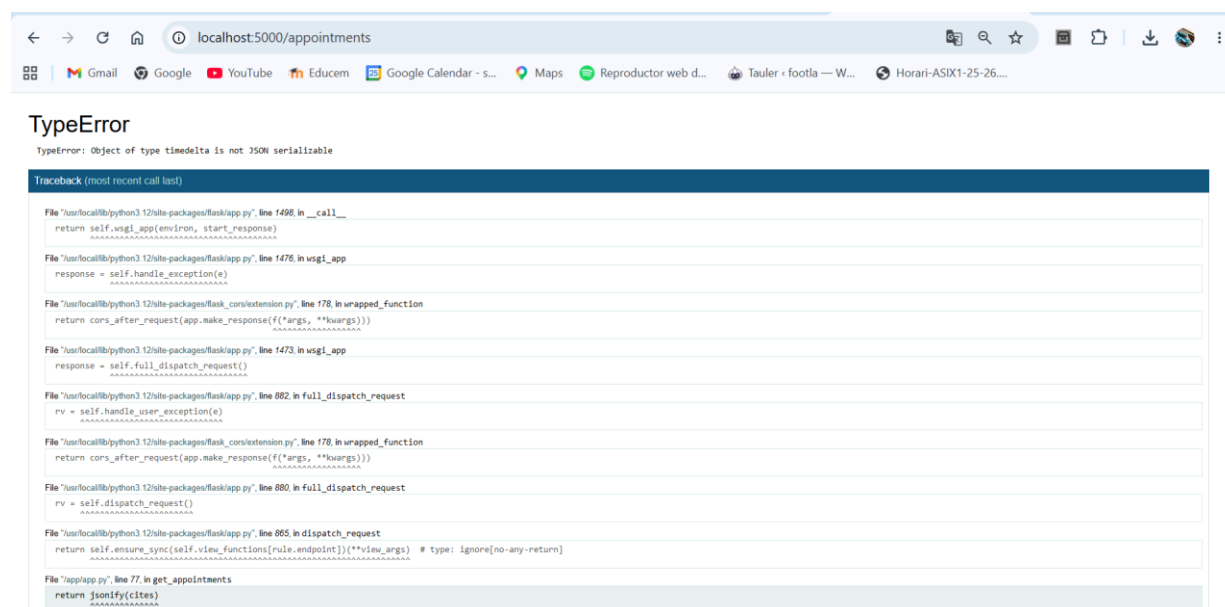


Figura 16. Error inicial de /appointments abans de corregir el format de l'hora.

Explicació curta del problema

La base de dades retornava l'hora com un tipus que Flask no podia convertir directament a JSON. La solució va ser passar aquest valor a text abans de retornar la resposta.

10. Conclusions i annexos

El projecte ha quedat funcionant amb els tres contenidors principals: web, API i base de dades. La web permet crear cites i consultar-les, l'API retorna dades en JSON i MariaDB guarda la informació.

El que més ha costat ha estat entendre on es comprovava cada part. La web es comprova al navegador, l'API també amb URLs, Docker amb Docker Desktop o `docker ps`, i el backup des de PowerShell. Quan ho separeu així, és més fàcil trobar els errors.

Com a millores futures, es podria afegir una zona d'administració, validar millor el formulari, poder canviar l'estat de les cites i adaptar els scripts perquè funcionin igual de bé en PowerShell i en WSL.

Resum final de proves

Prova	Comprovació	Resultat
Docker	<code>docker ps</code> mostra <code>taller_web</code> , <code>taller_api</code> i <code>taller_db</code> .	OK
Web	<code>localhost:8080</code> carrega la pàgina del taller.	OK
API health	<code>localhost:5000/health</code> retorna <code>api ok</code> i <code>database ok</code> .	OK
API vehicles	<code>localhost:5000/vehicles</code> retorna <code>vehicles</code> .	OK
API cites	<code>localhost:5000/appointments</code> retorna <code>cites</code> .	OK
Crear cita	La cita enviada des de la web queda registrada.	OK
Backup	Es genera un fitxer <code>.sql</code> dins <code>backups</code> .	OK

Comandes utilitzades

Acció	Comanda / URL
Arrencar el projecte	<code>docker compose up --build</code>
Veure contenidors	<code>docker ps</code>
Obrir la web	<code>http://localhost:8080</code>
Comprovar API	<code>http://localhost:5000/health</code>
Veure vehicles	<code>http://localhost:5000/vehicles</code>
Veure cites	<code>http://localhost:5000/appointments</code>
Crear backup	<code>docker exec taller_db mysqldump -uroot -pexample taller > backups/backup_taller.sql</code>
Aturar el projecte	<code>docker compose down</code>